

Homework 5

Due: 11:59pm on April 22, 2025 (Tuesday)

Objectives:

- Gain an experience to use Map interface to solve an interesting problem of comparing two documents.
- Try to build an application that meets your client's minimal acceptance tests and even more.

In Homeworks 1 and 3, you implemented `MyArray` and `SortedLinkedList` classes through which you were able to parse words from a text file. They have some of the useful capabilities and information.

Additionally, in Homework 4, you implemented `MyHashTable` class to store words and their frequencies from a file into a hash table so that you can search for words faster and see the frequency of words.

Moving on, it is about time for you to handle two files, not just one. A main question to consider in this homework assignment is:

How similar are two documents?

The similarities between documents are to be determined by the degree of the overlapping in contents of two documents. There are many different and complex algorithms to answer this question. In this homework, you are going to use an algorithm called *cosine similarity*.

It is a measure of similarity between two vectors by measuring the cosine of the angle between them and the algorithm has been used in search, text mining, and data mining. The cosine of the angle between two vectors basically determines whether two vectors are pointing in roughly the same direction or not. (http://en.wikipedia.org/wiki/Cosine_similarity)

$$similarity = \cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|}$$

where “.” denotes the dot-product of the two frequency vectors A and B, and $\|A\|$ denotes the length of a vector.

In this homework, you are to implement the following Java class:

```
public class Similarity
```

You must implement the following constructors and methods in your class:

```
public Similarity(String string)
public Similarity(File file)
public int numOfLines()    // runs in constant time
public “?” numOfWords()    // runs in constant time
```

```

public int numOfWordsNoDups() // runs in constant time
public double euclideanNorm()
public double dotProduct(Map<String, "> map)
public double distance(Map<String, "> map)
public Map<String, "> getMap()

```

If a String argument is passed when creating a Similarity object, you do not need to worry about number of lines. You can assume that it is one line with no special characters for new lines in the argument. The return type of the `numOfWords()` method is something you need to figure out to meet the requirements. Likewise, the data type of mapped values (marked with question marks above) in your map data structure is something you need to figure out to meet the requirements. Think carefully about what you need to be careful when `getMap()` method returns a reference to the Map instance in your class.

There are four steps you need to take care of to find out the cosine similarity between two texts or two files (documents).

Step 1: As in Homework 4, find out word frequency distribution for a text. Your class should have two constructors, one taking a string value and the other taking a file name. These constructors are in charge of building the data structures. And, using proper classes of the Java Collections Framework, you should store data into them. For example, if you read the string, "nice to meet you you look nice", then the first information you need is as follows:

```
{look=1, meet=1, nice=2, to=1, you=2}
```

When implementing the constructors in your `Similarity.java` file, please make sure to refer to `HW3Driver.java` file in relation to how to split text using any non-word character.

Step 2: Once you have words and their frequencies, the frequency of each word in the text should be used to find the Euclidean norm that is the length of the vector.

$$\|\mathbf{x}\| := \sqrt{x_1^2 + \cdots + x_n^2}$$

Step 3: Next step you need to take is to find out the dot-product of two frequency vectors X and Y as follows.

$$X = \{x_1, x_2, \dots, x_n\}; Y = \{y_1, y_2, \dots, y_n\};$$

$$X \cdot Y = x_1 y_1 + x_2 y_2 + \dots + x_n y_n$$

Step 4: Now you are ready to find out the cosine similarity or distance, `dist(D1, D2)`, of two texts or files as follows.

$$\text{dist}(D_1, D_2) = \arccos\left(\frac{\text{freq}(D_1) \cdot \text{freq}(D_2)}{\|\text{freq}(D_1)\| * \|\text{freq}(D_2)\|}\right)$$

A word is a sequence of letters [a..zA..Z], does not include digits [0..9], and does not include other characters than the sequence of letters [a..zA..Z]. Some examples of non-words include abc123, a12bc, 1234, ab_c, abc_. Gently ignore any non-words in your code. Unlike Homework 4, your application is in charge of converting all the words passed to you (either through a String object or a file) to lowercase.

Below are a few corner cases you might run into:

- The distance between two identical documents is 0.
- If two texts or documents do not have any common words, then the distance is $\pi/2$.
- When an argument is null or empty file, that means they do not have any common words. Thus, the distance is $\pi/2$.

For the constructor that takes a file parameter, because Autolab uses a different JVM than you have on your machine, please make sure to pass the second parameter of "latin1" when creating an instance of Scanner as follows.

```
new Scanner(file, "latin1");
```

To find out whether your program meets the requirements or not, your client has provided three test files along with the expected results for you. Your code must produce the EXACTLY SAME values of the expected output to pass the minimal acceptance tests. When running the test code with the provided text files, make sure to put the text files in the project (root) folder, not in the src (sub-)folder.

Be careful!

- When implementing the `dotProduct()` method, you need to **make sure it does not fall into quadratic running time on average**. Think carefully!
- Processing large files might lead to an incorrect distance between two documents. Think about why! Your code must be able to handle any given input file regardless of its size (extremely large and maybe with many duplicate words, etc.). Even if it may not seem to happen with realistic random datasets, what your client is asking here is for you to be **EXTREMELY defensive** here, and your code should reflect it.
- Make sure to use proper constants and methods in `java.lang.Math` class.

Deliverables:

- Submit your `Similarity.java` file using Autolab (<https://autolab.andrew.cmu.edu>). Do not zip it. Do not use package.
 - Using either single-line or multi-line comments, be sure to mention what data structures from the Java Collections Framework you chose and for what purpose.
 - Also, at the top of your `dotProduct` method, clearly mention why your implementation does not fall into the quadratic running time complexity on average.

Grading:

Autolab will grade your assignment as follows.

- Working code: 90 points
- Coding conventions: 10 points
 - We will deduct one point for each coding convention issue detected.

In case you do not pass test case(s), please spend some time to think about edge cases you may have missed and test thoroughly before asking questions.

Autolab will show you the results of its grading within approximately a few minutes of your

submission. You may submit multiple times to correct any problems with your assignment. Autolab uses the last submission as your grade.

The Autolab score is not final. The TAs will look at your source code for correctness and design, and deduct points if applicable. The most important criterion is always correctness. It is also critical that your code is efficient and follows the specification properly. Additionally, it should be readable and well organized. This includes proper use of clear comments. Points will be deducted for poor design decisions and unreadable code, etc. Common sources of deductions from past semesters are shown below. Note that this list is not exhaustive and is merely intended to draw your attention to parts easy to overlook with this assignment; you may lose points if the TAs find substantial issues with your code outside of this list.

-
- 10 pts
 - Having the dotProduct method be $O(n^2)$ on average
 - 5 pts
 - Using an outdated class in the Java Collections Framework to store data (e.g., Hashtable)
 - Not explaining why the dotProduct method is not $O(n^2)$ on average
 - Returning a direct reference in the getMap method without making a defensive copy
 - Being unable to correctly handle very large files
 - Having non-private helper methods, or a non-public constructor
 - Having non-private instance variables
 - Not having meaningful Javadoc comments
 - Having unaddressed warnings in code
 - Not removing dead code
 - 3 pts
 - Not explaining the choice of data structures from the Java Collections Framework

As mentioned in the syllabus, **late submissions will NOT be accepted**. If you have multiple versions of your code file, make sure you submit the correct version.